Design Patterns - Lab 4
Chain of Responsibility and Command

Goal

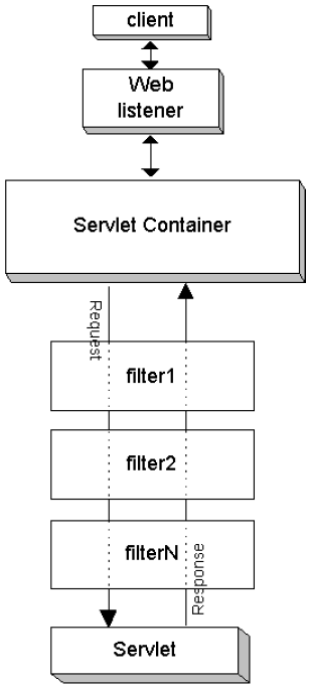
This laboratory discusses two patterns for decoupling processing. An action can be triggered from different places and can / should be handled by different processors. The Chain of Responsibility and Command patterns allow segregating business logic across multiple objects.

We will be building a RESTful Web service for managing books through an endpoint that will support the following methods:

- GET /books - returns all the books
- GET /books/{id} - returns details about the book with specified {id}
- POST /books - creates a new book
- PUT /books/{id} - replaces the representation of specified book
- DELETE /book/{id} - deletes the book with specified {id}

Tasks

1. We need to log (to a logfile) each request, before and after actual processing happens. And we don't want to write code for each controller method.



2. This is how HTTP requests are processed by the application container. So, what would be a good candidate for us to hook our logging responsibility?

……..

3. This is an example of ……. design pattern.

4. What is the trigger for all the actions that we need to support in our Web service? What is the entry point?

……..
……..
……..

5. Create the BooksController that handles the five HTTP methods from above (for now, they would do nothing or return some dummy values). This is the presentation layer of your backend.

6. Create the RequestLoggingFilter. Implementation hint: see the following post https://www.baeldung.com/spring-boot-add-filter or the filter example in Lecture 8-9

7. How can we decouple the business logic from the controller? Why is it not a good practice to include logic inside the controller layer? What OOP principle is violated?

   ……
   ……

   (yeah, a service layer is an excellent idea for all the reasons you've mentioned; and yes, SRP is the infringed principle)

8. In most of the cases, you'd start developing your service layer…. But wait….. not so fast Turbo (2013). Let's think a bit about processing types:
   a. synchronous processing
   b. asynchronous processing

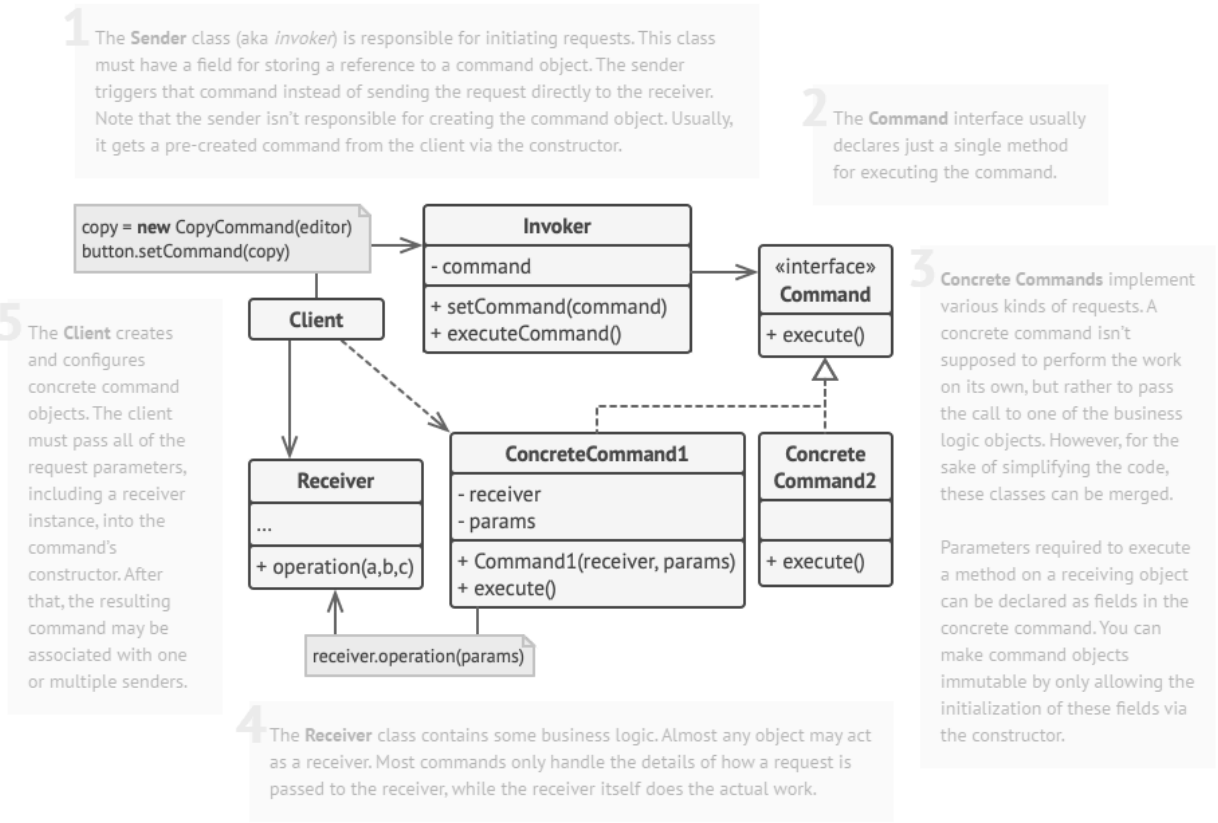
   What type of processing is appropriate for the following methods/requests?
   Hint: assume that creating a new book requires the system to obtain a valid ISBN from an external service, with an uncommitted Service Level Agreement.

- GET /books ……..
- GET /books/{id} …….
- POST /books …….
- PUT /books/{id} …….
- DELETE /book/{id} …….

9. How can we decouple the trigger and processing of a particular action from the action (business logic) itself? As noticed above, some actions/requests are more suitable for synchronous processing, while others are better handled asynchronously. In time, we may learn that we'll need to change the processing type, while business logic remains the same.

……
……
……
……

10. In this situation, the Command design pattern is a reliable friend:



Source: https://refactoring.guru/design-patterns/command

11. What interfaces and classes should we define and implement?
    The Client is ……..
    The Invoker is ……
    The Receiver is ……
    The Commands are: ……
    ……
    ……
    ……

12. Implement the five methods for /books endpoint using the Command pattern

a. Assume for now only synchronous execution of the commands
b. Commands will make the top-level services layer of your application and will delegate to downstream services (e.g. BooksService)
c. Commands may implement orchestration logic for more complex tasks, when two or more dependent services are involved

13. Implementation hints
    - Controller constructor is configured with commands' external dependencies (e.g. BooksService) using dependency injection
    - The endpoint method creates the command based on the received data
        - pass the external dependencies (e.g., BooksService) in the CommandContext object
        - (alternatively, all arguments, incl. external dependencies, can be injected in the Command's constructor)
    - For now, the BooksService can manage the objects using an in-memory data structure, such as a list/map; we will connect this service to the persistence layer in the next labs.

14. What about asynchronous processing of requests we have identified as being suitable for this processing type? How can we process our commands in an asynchronous way?
    a. Approach 1: using template method pattern, implement a base SynchronousCommand, respectively AsynchronousCommand (that would call the execute method in synchronous/asynchronous way), and derive the concrete commands from the appropriate base class
    b. Approach 2: introduce a CommandExecutor with two distinct implementations (one synchronous and another one asynchronous) and use it as *invoker* / *command executor* (see approach discussed in lecture 10)

15. Choose your preferred approach and refactor the implementation to account for synchronous vs. asynchronous request processing. Explain your choice.

……
……
……
……
……

16. Implementing asynchronous processing
    a. The request reaches the endpoint method
    b. The code **queues the request** and responds with HTTP code **202 (Accepted)** to the caller providing a form of details (such as ID) that would allow caller to retrieve later the status of the accepted request
    c. The back-end code uses the **Thread pool** pattern, which is made available in Executors.newFixedThreadPool(), to pick-up requests from the queue and **process** them
    d. The **client polls** the status of the accepted request through a GET method in order to determine whether the request processing has been completed.

17. How does your class diagram look now? Draw it below

